

# 5. 합성곱 신경망 I

AI/데이터

2025/09/20 20:57

<http://blog.naver.com/sirutio03/224015560254>

## 5.1 합성곱 신경망

이미지 전체를 한 번에 계산하지 않고, 국소적 부분을 계산해 시간과 자원을 절약.(세밀한 부분까지 분석)

- 합성곱층의 필요성

그림을 펼쳐서 각 픽셀에 가중치를 곱하여 은닉층으로 전달하면 데이터의 공간적 구조를 무시하게 됨.

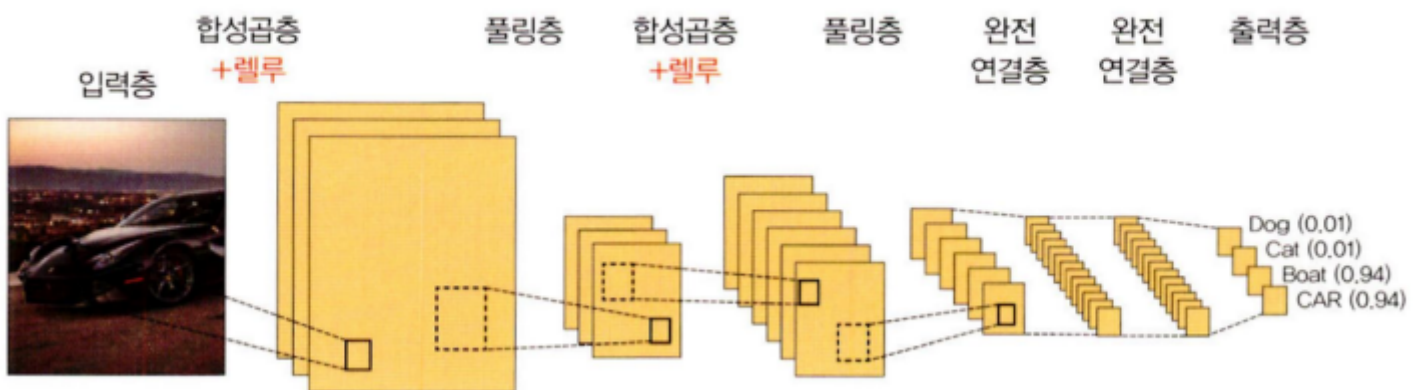
이를 방지하려 도입

- 합성곱 신경망 구조

합성곱, 풀링층을 거치며 이미지 주요 특성 벡터 추출, 완전연결층 거치면서 1차원 벡터로 변환.

출력층에서 소프트맥스 함수를 사용해 최종 결과 출력.

▼ 그림 5-2 합성곱 신경망 구조



### 1. 입력층

: 입력 이미지 데이터가 최초로 거치는 계층, 이미지는 height/width/channel 값을 갖추는 3차원 데이터로, 채널값은 그 레이스케일은 1, 컬러면 3을 가짐.

### 2. 합성곱층

: 입력 데이터에서 특성 추출하여 특성 맵 완성

1단계. 입력 이미지에 3x3 필터(커널) 적용

2단계. 필터가 스트라이드(1)만큼 이동

3단계 ... 이를 반복하며 마지막까지 이동하여 feature map 채움

+ 컬러 이미지 합성곱은 RGB 각각에 서로 다른 가중치로 합성곱을 적용한 후 결과를 더해 줌.

▼ 그림 5-4 입력 이미지에 3×3 필터 적용



▼ 그림 5-5 입력 이미지에 필터가 1만큼 이동

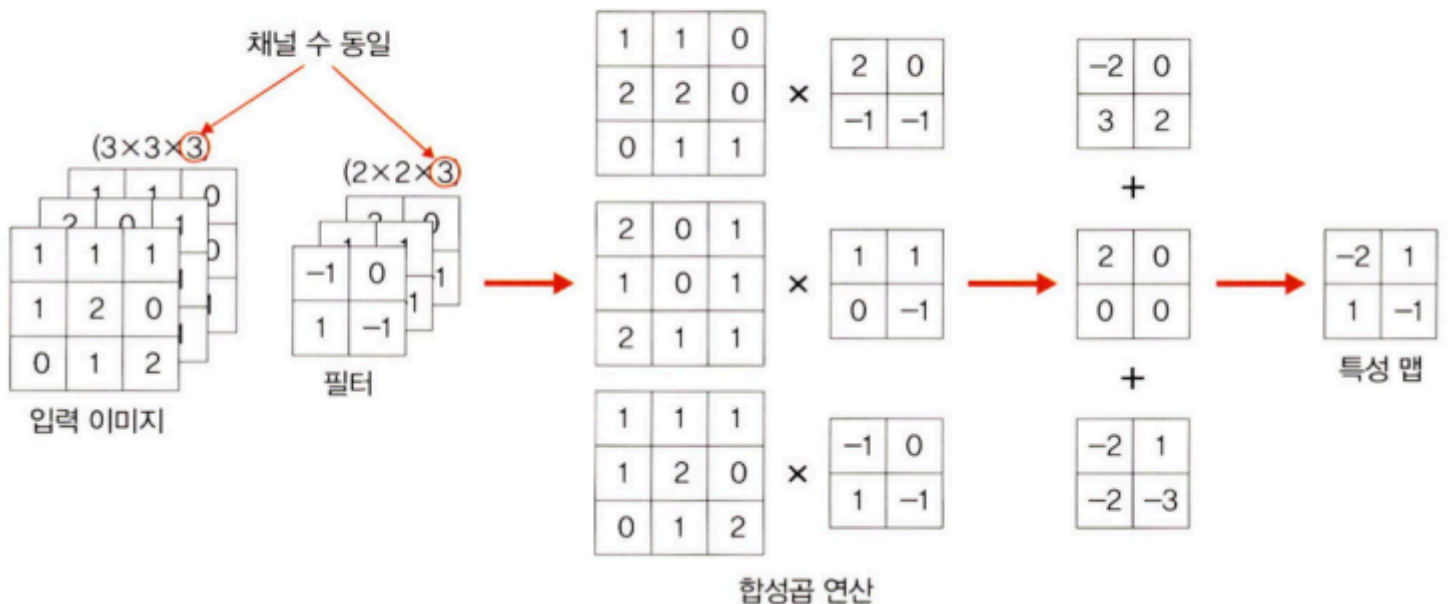


▼ 그림 5-9 입력 이미지에 필터가 1만큼 마지막으로 이동



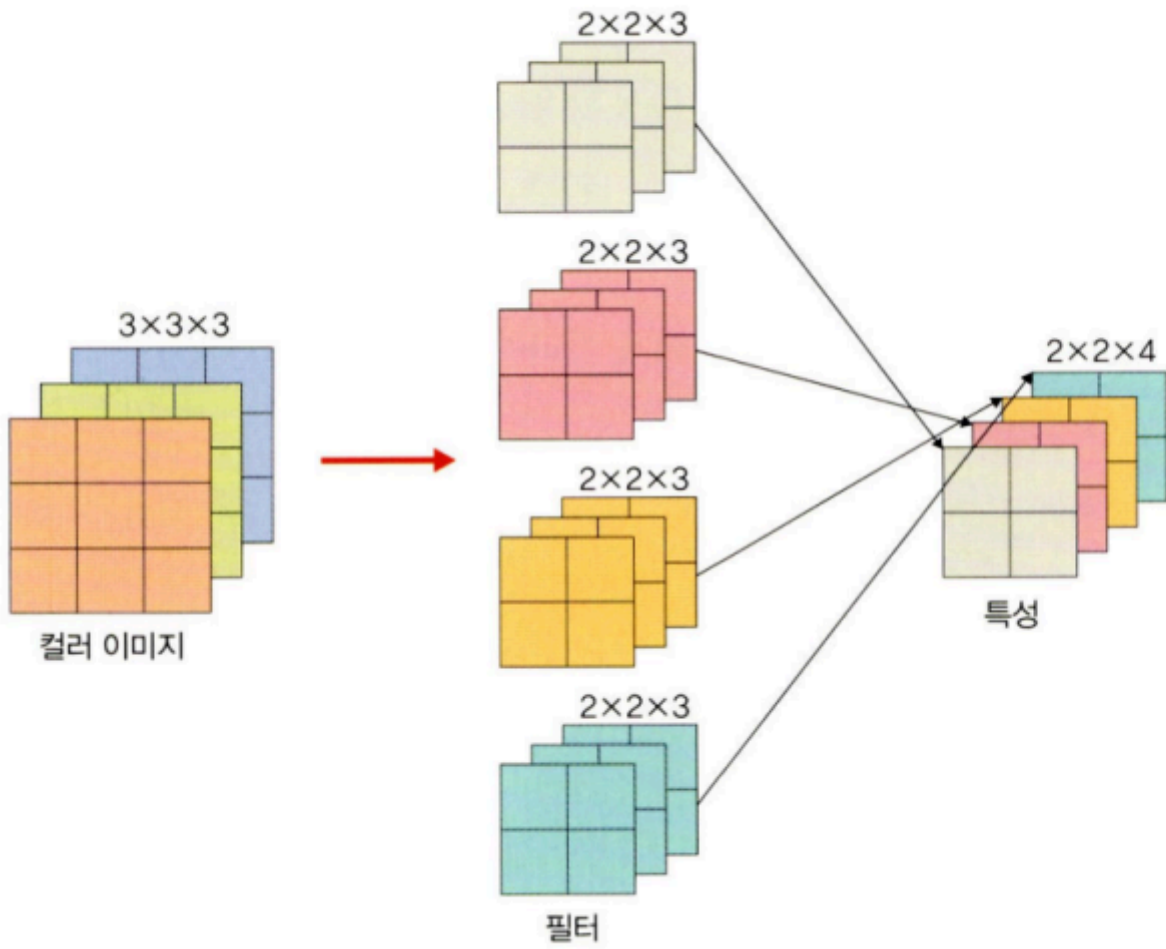
2-1. 그레이스케일: 합성곱층에서 필터가 이동하며 특성 맵을 완성하는 과정

▼ 그림 5-10 컬러 이미지 합성곱



2-2. 컬러 이미지: 그레이스케일과 동일하나 RGB마다 가중치가 다름

♥ 그림 5-11 필터가 2 이상인 합성곱



- 입력 데이터:  $W_1 \times H_1 \times D_1$  ( $W_1$ : 가로,  $H_1$ : 세로,  $D_1$ : 채널 또는 깊이)
- 하이퍼파라미터
  - 필터 개수:  $K$
  - 필터 크기:  $F$
  - 스트라이드:  $S$
  - 패딩:  $P$
- 출력 데이터
  - $W_2 = (W_1 - F + 2P) / S + 1$
  - $H_2 = (H_1 - F + 2P) / S + 1$
  - $D_2 = K$

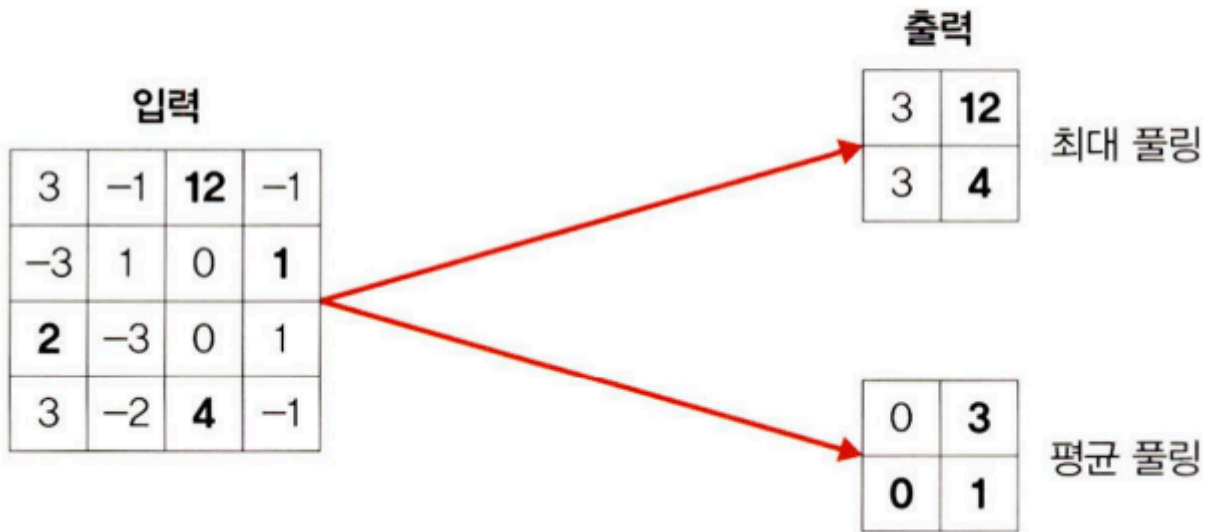
필터가 여러 개인 경우 특성 맵의 채널이 늘어남

### 3. 풀링층

: 특성 맵의 차원을 다운 샘플링하여 연산량을 감소시키고, 주요한 특성 벡터를 추출

1. 최대 풀링: 대상 영역에서 최대값을 추출
2. 평균 풀링: 대상 영역에서 평균을 반환(각 커널 값을 평균화 시켜 중요한 가중치를 갖는 값 희미해짐)

▼ 그림 5-17 최대 풀링과 평균 풀링 비교



- 입력 데이터:  $W_1 \times H_1 \times D_1$
- 하이퍼파라미터
  - 필터 크기:  $F$
  - 스트라이드:  $S$
- 출력 데이터
  - $W_2 = (W_1 - F) / S + 1$
  - $H_2 = (H_1 - F) / S + 1$
  - $D_2 = D_1$

최대 풀링과 평균 풀링

#### 4. 완전연결층

3차원 벡터에서 1차원 벡터로 펼쳐짐

### 5. 출력층

소프트맥스 활성화 함수가 사용되어 이미지가 각 레이블에 속할 확률값이 출력됨

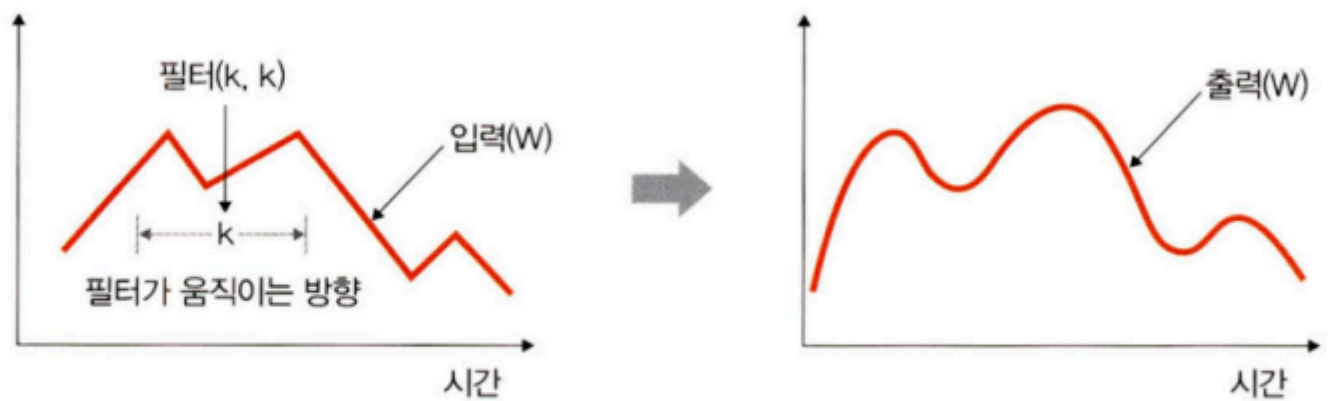
- 1D 합성곱

: 필터가 시간을 축으로 좌우로만 이동할 수 있는 합성곱, 출력 형태가 1D형태의 배열

- 2D 합성곱

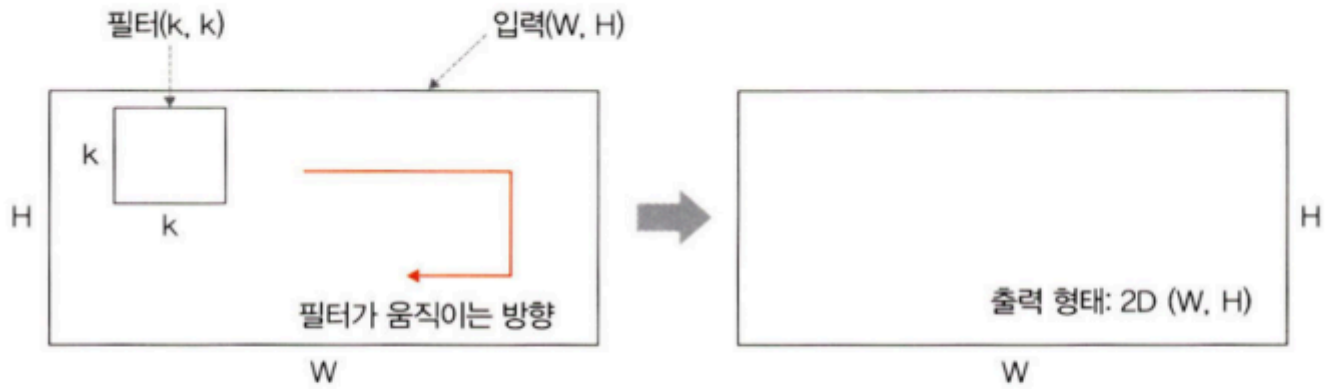
: 필터가 방향 두 개로 움직여 2D 행렬 형태로 출력하게 됨

▼ 그림 5-19 1D 합성곱



- 입력: W 너비(Width)
- 필터:  $k \times k$  (높이  $\times$  너비)
- 출력: W 너비(Width)

♥ 그림 5-20 2D 합성곱



- 입력: W 너비(Width), H 높이(Height)
- 필터:  $k \times k$  (높이  $\times$  너비)
- 출력: W 너비(Width), H 높이(Height)

1D 합성곱, 2D 합성곱

- 3D 합성곱

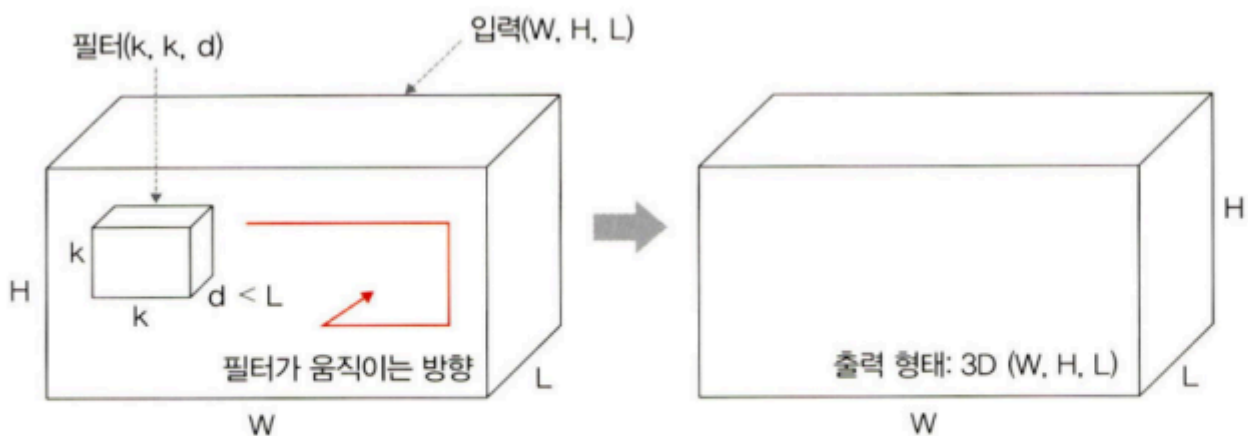
: 필터가 방향 세 개로 움직여 3D 형태로 출력, 입력(W,H,L)에 대해 필터 (k,k,d)를 적용했을 때  $d < L$ 을 유지해야 함

- 3D 입력을 갖는 2D 합성곱

: 필터에 대한 길이 L이 입력 채널의 길이 L과 같아야 하기 때문에 만들어지는 합성곱 형태

ex) LeNet-5, VGG

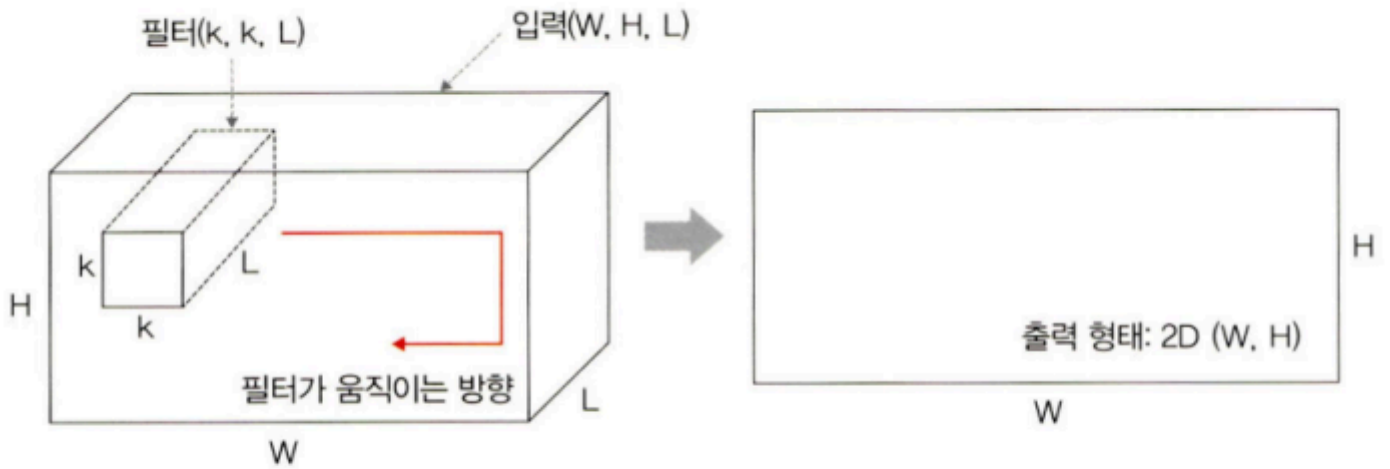
♥ 그림 5-21 3D 합성곱



- 입력: W 너비(Width), H 높이(Height), L 길이(Length)
- 필터:  $k \times k \times d$  (높이  $\times$  너비), d: 깊이(depth)
- 출력: W 너비(Width), H 높이(Height), L 길이(Length)



▼ 그림 5-22 3D 입력을 갖는 2D 합성곱



- 입력:  $W$  너비(Width),  $H$  높이(Height),  $L$  길이(Length)
- 필터:  $k \times k$  (높이  $\times$  너비),  $L$  길이(Length)
- 출력:  $W$  너비(Width),  $H$  높이(Height)

3D 합성곱과 3D 입력을 갖는 2D 합성곱

- 1 x 1 합성곱

( $W, H, L$ ) 형태로 입력됨, 필터가 ( $1, 1, L$ )로 적용. 채널 수를 조정해서 연산량이 감소됨. ( $W, H$ )로 출력

ex) GoogLeNet

## 5.2 합성곱 신경망 맛보기

- fashion\_mnist 데이터셋: 토치비전에 내장된 예제 데이터로, 작은 이미지들의 모음. 훈련 데이터는 넘파이 배열, 레이블 데이터는 0부터 9까지의 정수값을 갖는 배열.(옷 종류 클래스)
- GPU 사용

```
# 하나의 GPU 사용 device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu") model = Net()
model.to(device) # 다수의 GPU 사용 device = torch.device("cuda" if torch.cuda.is_available() else "cpu") model
= Net() if torch.cuda.device_count() > 1: model = nn. DataParallel(net) model.to(device)
```

- fashion\_mnist 데이터셋 내려받기

```
torchvision.datasets.FashionMNIST("위치 지정", download=True, transform=)
```

파라미터 설명

> download: True일 시 위치에 해당 데이터셋 있는지 확인 후 내려받음

> transform: 이미지를 텐서(0~1)로 변경

- 데이터로더 전달

: 내려받은 데이터를 메모리로 불러오기 위한 코드

`torch.utils.data.DataLoader(데이터를 불러올 데이터셋, batch_size=100)`

파라미터 설명

> batch\_size: 데이터를 배치로 묶어줌

- 분류에 사용될 클래스 정의

`random.rand` vs `random.randn` vs `random.randint` `np.random.rand(8)` # 0~1 사이의 정규표준분포 난수를 1x8의 행렬로 출력 `np.random.randint(10)` # 0~10의 임의 숫자 출력 `np.random.randn(4,2)` # 평균이 0이고, 표준편차가 1인 가우시안 정규분포 난수를 4x2의 행렬로 출력

20개의 랜덤 이미지가 시각적으로 표현됨

심층 신경망 모델 생성

`nn`: 딥러닝 모델 구성에 필요한 모듈이 모여 있는 패키지

`nn.Linear()`: 선형 회귀 모델

`nn.Dropout(p)`: p만큼의 비율로 텐서값 0, 0이 되지 않는 값은 기존값에  $(1/(1-p))$ 만큼 곱해져 커짐

`forward()`: 순전파 연산 진행 함수

`.view()`: 넘파이의 `reshape`와 같은 역할

`F.relu()`: `forward()`에서 정의 / `nn.ReLU()`: `__init__()`에서 정의

▼ 표 5-1 `nn.xx`와 `nn.functional.xx`의 사용 방법 비교

| 구분    | <code>nn.xx</code>                                                  | <code>nn.functional.xx</code>                                                             |
|-------|---------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| 형태    | <code>nn.Conv2d</code> : 클래스<br><code>nn.Module</code> 클래스를 상속받아 사용 | <code>nn.functional.conv2d</code> : 함수<br><code>def function (input)</code> 으로 정의된 순수한 함수 |
| 호출 방법 | 먼저 하이퍼파라미터를 전달한 후 함수 호출을 통해 데이터 전달                                  | 함수를 호출할 때 하이퍼파라미터, 데이터 전달                                                                 |
| 위치    | <code>nn.Sequential</code> 내에 위치                                    | <code>nn.Sequential</code> 에 위치할 수 없음                                                     |
| 파라미터  | 파라미터를 새로 정의할 필요 없음                                                  | 가중치를 수동으로 전달해야 할 때마다 자체 가중치를 정의                                                           |

활성화 함수 정의 사용 방법 차이

```
class FashionDNN(nn.Module):  
    def __init__(self):  
        super(FashionDNN, self).__init__() # FashionDNN 클래스 상
```

```

속 받음 self.fc1 = nn.Linear(in_features=784, out_features=256) # in_features: 입력 크기, out_features: 출력 크
기 self.drop = nn.Dropout(0.25) self.fc2 = nn.Linear(in_features=256, out_features=128) self.fc3 =
nn.Linear(in_features=128, out_features=10) def forward(self, input_data): # 실제 데이터 연산 진행됨, 순전파
학습 진행 out = input_data.view(-1, 784) # 넘파이의 reshape와 같은 역할 out = F.relu(self.fc1(out)) # 활성화 함
수 정의 out = self.drop(out) out = F.relu(self.fc2(out)) out = self.fc3(out) return out

```

### 심층 신경망에서 필요한 파라미터 정의

경사 하강법은 Adam 사용(torch.optim.Adam(model.parameters(), lr=learning\_rate));

### 심층 신경망을 이용한 모델 학습 및 성능 평가

```

images, labels = images.to(device), labels.to(device) # 모델과 데이터는 동일한 장치에 있어야 함,
model.to(device)와 같은 위치 train = Variable(images.view(100, 1, 28, 28)) # Autograd 패키지 사용, Variable()
사용해서 역전파를 위한 미분 값 자동으로 계산 classification accuracy = correct predictions / total predictions # 분
류 문제에 대한 정확도

```

### 합성곱 네트워크 생성

nn.Sequential: 계층을 차례로 쌓을 수 있도록 수식과 활성화 함수를 연결해 주는 역할, 하나의 컨테이너에 여러 개의 계층을 구현하는 방법

nn.Conv2d(): 합성곱층 추출 코드, in\_channels=입력 채널 수, out\_channels=출력 채널 수, kernel\_size=커널 크기 (필터), padding=패딩 크기, 출력 크기 조정 위해 입력 데이터 주위에 0을 채움. 패딩 값이 클수록 출력 크기도 커짐.

nn.BatchNorm2d(): 각 배치 단위별로 데이터가 다양한 분포를 가져도 평균과 분산을 이용해 정규화하는 것

nn.MaxPool2d(): 이미지 크기를 축소시키는 용도, 풀링 계층 처리.

nn.Linear 사용해서 데이터 형태 1차원으로 변경(합성곱층->완전연결층)

```

class FashionCNN(nn.Module): def __init__(self): super(FashionCNN, self).__init__() self.layer1 =
nn.Sequential( nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1),
nn.BatchNorm2d(32), nn.ReLU(), nn.MaxPool2d(kernel_size=2, stride=2) ) self.layer2 = nn.Sequential(
nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3), nn.BatchNorm2d(64), nn.ReLU(), nn.
MaxPool2d(2) ) self.fc1 = nn.Linear(in_features=64*6*6, out_features=600) self.drop = nn.Dropout2d(0.25)
self.fc2 = nn.Linear(in_features=600, out_features=120) self.fc3 = nn.Linear(in_features=120,
out_features=10) def forward(self, x): out = self.layer1(x) out = self.layer2(out) out = out.view(out.size(0),
-1) # out.size(0) = 100을 의미 out = self.fc1(out) out = self.drop(out) out = self.fc2(out) out = self.fc3(out)
return out

```

## Conv2d 계층에서의 출력 크기 구하는 공식

- 출력 크기  $= (W - F + 2P) / S + 1$ 
  - $W$ : 입력 데이터의 크기(input\_volume\_size)
  - $F$ : 커널 크기(kernel\_size)
  - $P$ : 패딩 크기(padding\_size)
  - $S$ : 스트라이드(strides)

예를 들어 첫 번째 Conv2d 계층은 다음과 같습니다.

### MaxPool2d 계층에서의 출력 크기 구하는 공식

- 출력 크기  $= IF / F$ 
  - $IF$ : 입력 필터의 크기(input\_filter\_size, 또한 바로 앞의 Conv2d의 출력 크기이기도 합니다)
  - $F$ : 커널 크기(kernel\_size)

완전연결층으로 보내기 위해 1차원 변경 시 출력 크기 구하는 공식

합성곱 네트워크를 위한 파라미터 정의, 모델 학습 및 성능 평가

딥러닝 신경망과 동일

=> 비교 결론: 심층 신경망과 비교하여 정확도가 약간 높음(86 vs 88)

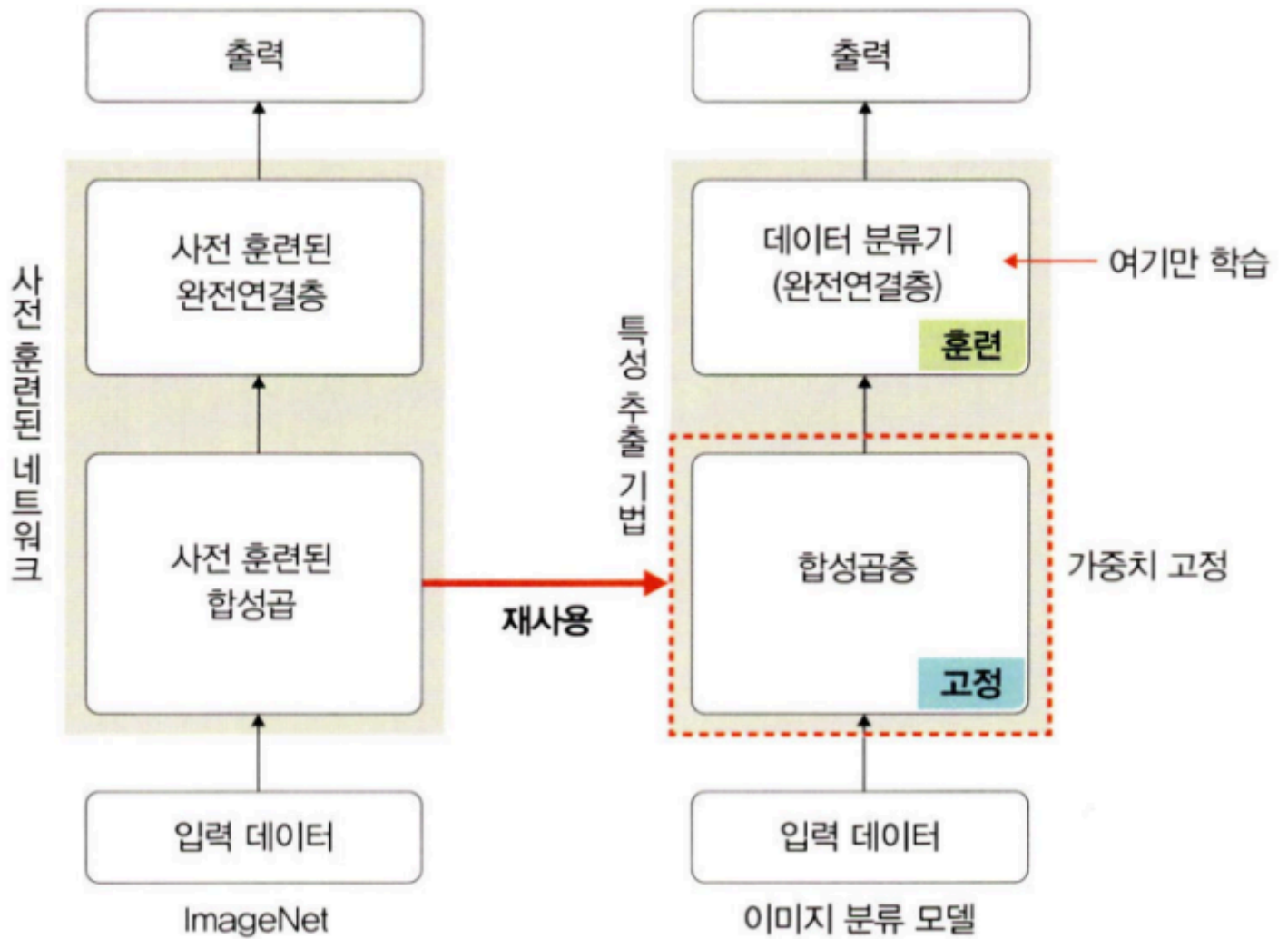
---

### 5.3 전이 학습

- 전이 학습: 아주 큰 데이터셋을 써서 훈련된 모델(네트워크)의 가중치를 가져와 우리가 해결하려는 과제에 맞게 보정해서 사용
- 특성 추출 기법: ImageNet 데이터셋으로 사전 훈련된 모델을 가져온 후 완전연결층만 새로 만들. 이미지 카테고리

리 결정 부분만 학습하고 나머지 계층들은 학습되지 않도록 함. -> 네트워크의 합성곱층에 새로운 데이터를 통과시키고, 해당 출력을 데이터 분류기(완전연결층)에서 훈련시킴.

▼ 그림 5-30 특성 추출 기법



- Xception
- Inception V3
- ResNet50
- VGG16
- VGG19
- MobileNet

특성 추출 기법과 사용 가능한 이미지 분류 모델

라이브러리 호출 -> 훈련 데이터 전처리

`torchvision.transform`(이미지 데이터를 변환하여 네트워크 입력으로 사용)

> `transform.Resize([256,256])`: 크기 조정, 합성곱층 통과 위한 전처리 과정 `.RandomResizedCrop(224)`: 랜덤한 크기 및 비율로 잘라 데이터 확장 용도로 사용 `.RandomHorizontalFlip()`: 이미지를 랜덤하게 수평으로 뒤집음

`.ToTensor()`: 이미지 데이터 텐서로 변환 > `datasets.ImageFolder('불러올 데이터가 위치한 경로',`

`transform=transform)` > `torch.utils.data.DataLoader`(데이터셋 지정, `batch_size`=한 번에 불러올 데이터 크기, `num_workers`=데이터 불러올 때 사용할 하위 프로세스 개수, `shuffle`=무작위로 섞을거면 `True`)

+ `RandomResizedCrop`의 데이터 확장 과정에 대해 알아보기(204-207p)

학습에 사용될 이미지 출력

: `next(iter(train_loader))`로 `samples`, `labels`값을 하나씩 꺼내서 저장 [책에서는 `.next()` 사용]

클래스 `cat`, `dog`로 나눈 후 레이블 정보(클래스)를 함께 출력

사전 훈련된 모델 내려받기(`ResNet18`)

: 50개의 계층으로 구성된 합성곱 신경망, `pretrained=True`

사전 훈련된 모델의 파라미터 학습 유무 지정

: `param.requires_grad = False`

완전연결층 추가

: `nn.Linear(512,2)`, 2는 클래스가 두 개라는 의미

모델 객체 생성 및 손실 함수 정의

: `.resnet18()`, 합성곱층 가중치는 고정하고(`requires_grad=False`) 완전연결층은 학습(`requires_grad=True`), 손실 함수 정의(`CrossEntropyLoss()`)

모델 학습을 위한 함수 생성, 파라미터 학습 결과를 옵티마이저에 전달

### 훈련 데이터로 모델 학습

13 에포크로 결과 출력 -> 약 93%의 높은 Best Acc

### 테스트 데이터 전처리

테스트 데이터 평가 함수 생성(앞서 훈련 데이터 평가에 사용했던 모델 사용/saved\_models)

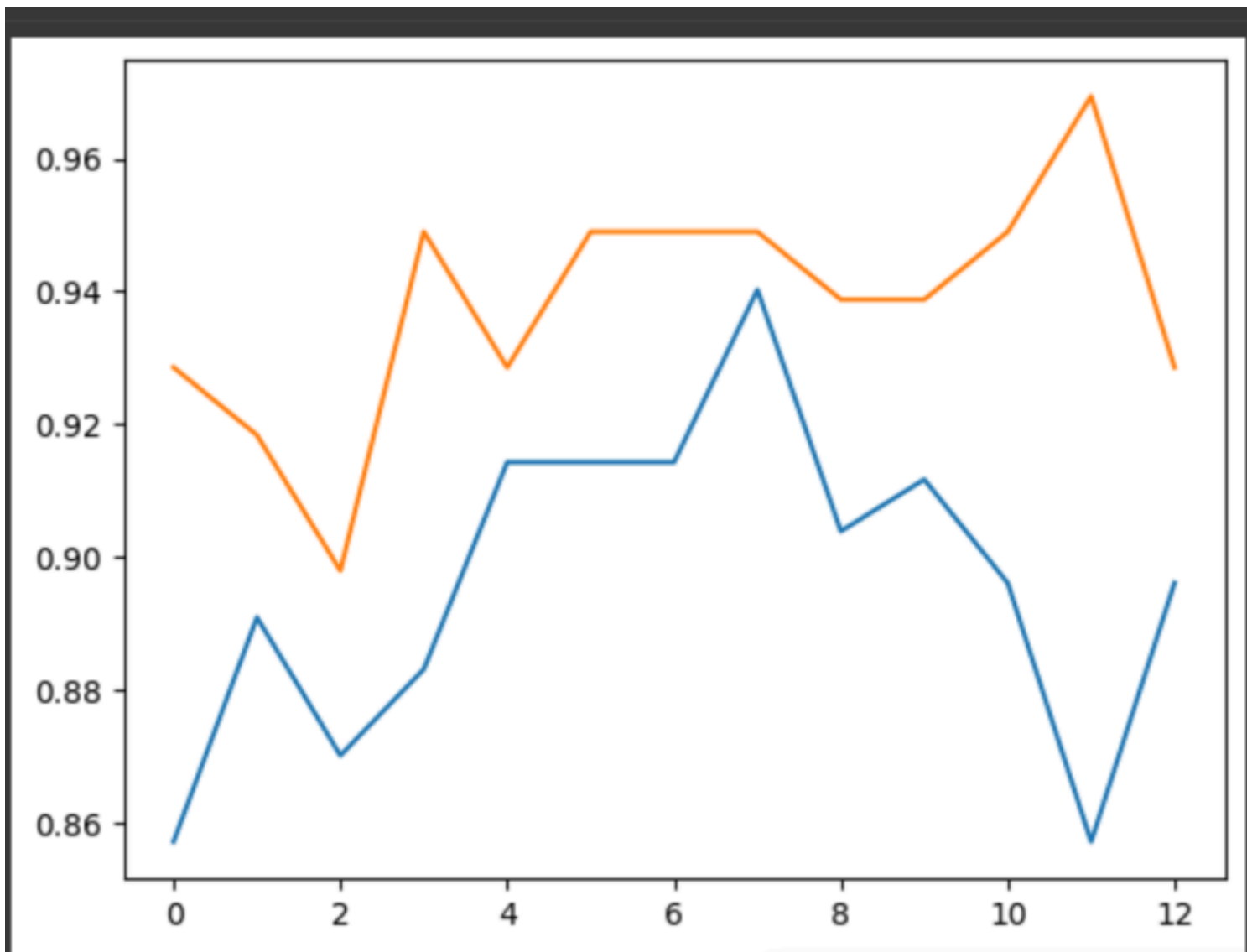
: torch.max = 주어진 텐서 배열의 최댓값이 들어 있는 index 반환, preds.qu(labels)는 preds배열과 labels가 일치하는지 검사 용도.

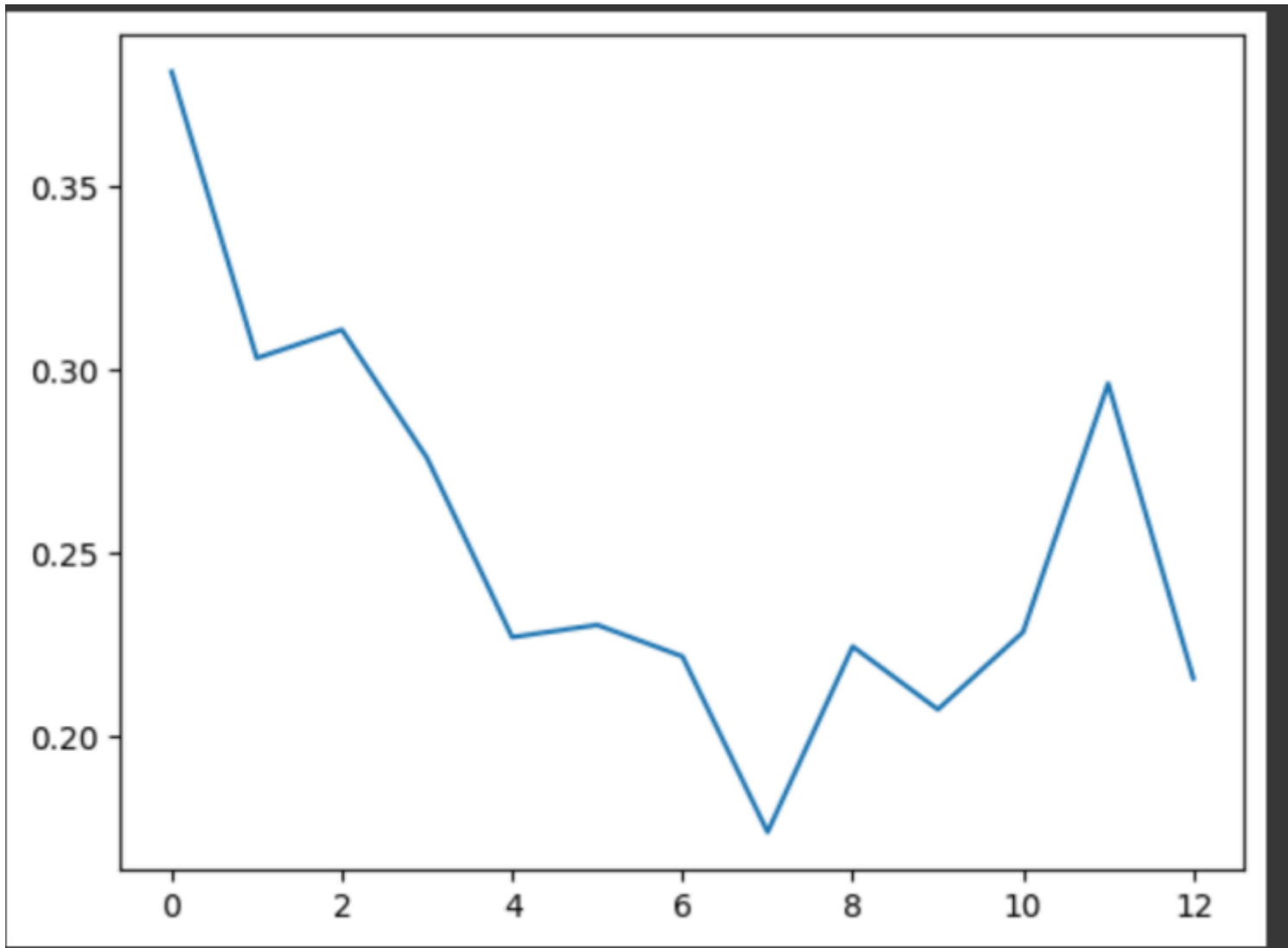
테스트 데이터 평가 함수에 적용

: eval\_model(resnet18, test\_loader, device)

=> 모델의 네트워크를 직접 개발자가 구현하고 최적의 파라미터 값을 찾는다면 시간이 오래 걸려, 전이 학습은 실무에서도 많이 사용됨.

훈련 & 테스트 데이터 정확도 및 훈련 데이터 오차 그래프로 확인





주황-테스트, 파랑-훈련 데이터. 좌측 정확도 / 우측 오차

실제 예측 진행

예측 이미지 전처리 함수 생성

: `tensor.clone()` = 기존 텐서 내용 복사, `detach()` = 기울기 전파 X, `clip()` = 입력 값 특정 범위로 적용

+ 계산 그래프, 연쇄 법칙(225p)

테스트 데이터셋 이용하여 개 고양이 예측 결과 출력

: `add_subplot(행 수, 열 수, 인덱스, tick을 삭제)`, 예측값이 0이면 고양이 1이면 개로 출력

+ tick이란 x축 y축으로 나뉘어짐. (20, 0.7)

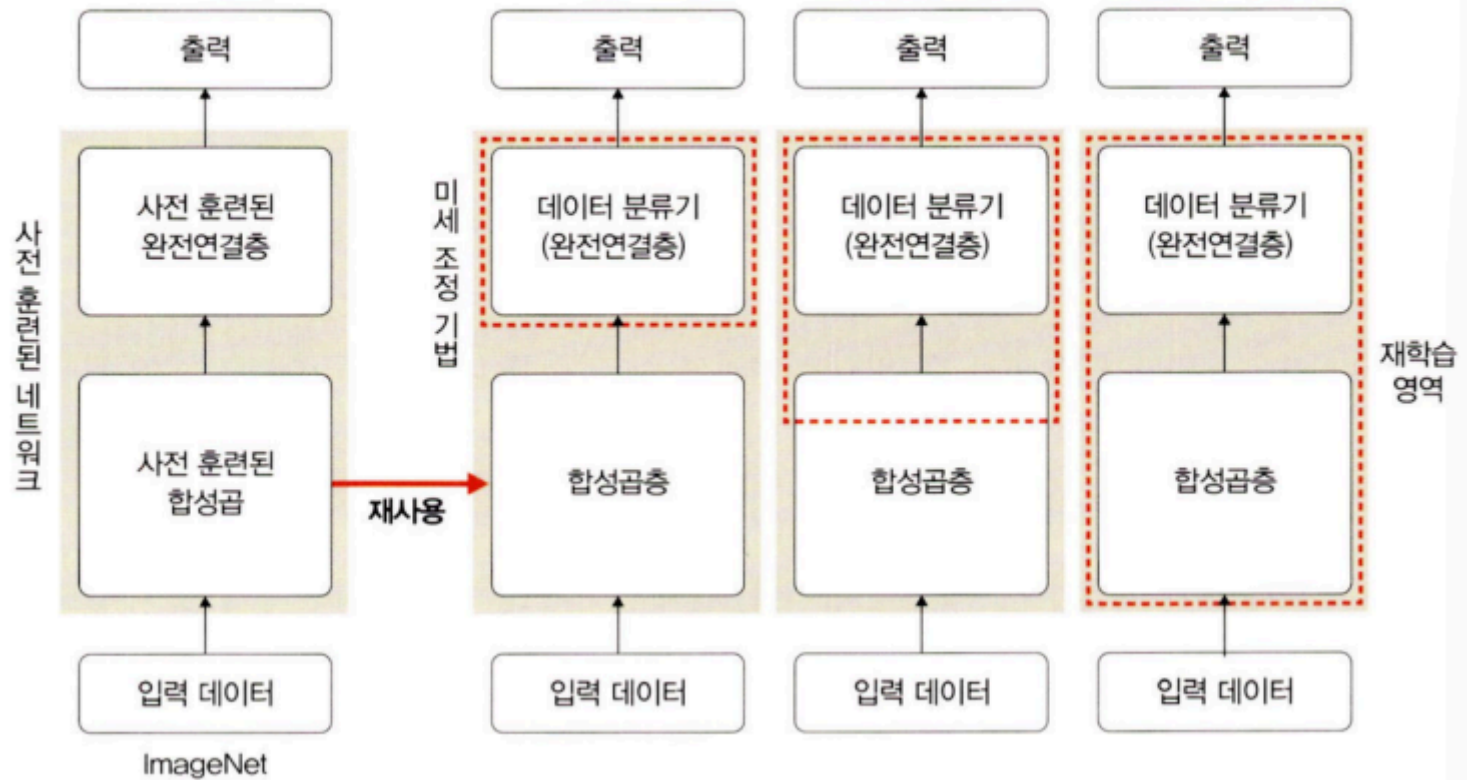
- 미세 조정 기법: 사전 훈련된 모델과 합성곱층, 데이터 분류기의 가중치를 업데이트하여 훈련시키는 방식, 사전 학습된 모델을 목적에 맞게 재학습시키거나 학습된 가중치의 일부를 재학습시킴, 연산량 많음.

#### 1. 데이터셋 크고 네트워크와 유사성 작음: 모델 전체 재학습



2. 데이터셋 크고 네트워크와 유사성 큼: 합성곱층 뒷부분, 데이터 분류기 재학습
3. 데이터셋 작고 네트워크와 유사성 작음: 합성곱층 일부분과 데이터 분류기 학습
4. 데이터셋 작고 네트워크와 유사성 큼: 데이터 분류기만 학습

▼ 그림 5-44 미세 조정 기법



## 5.4 설명 가능한 CNN

딥러닝 처리 결과를 사람이 이해할 수 있는 방식으로 제시하는 기술.

- 특성 맵 시각화: 필터를 입력에 적용한 결과.

라이브러리 호출(pillow 설치) -> 설명 가능한 네트워크 생성(렐루 활성화 함수 사용) -> 객체화(XAI)

+ 로그 소프트맥스 설명(231p)

특성 맵을 확인하기 위한 클래스 정의, 이미지 호출

: 파이토치의 hook 기능 사용하여 각 계층의 활성화 함수 및 기울기 값 확인(중간 결과값 알 수 있음)

Conv2d 특성 맵 확인(시각적 표현)

: 첫 번째 계층에서 입력 이미지 형태가 많이 유지됨을 관찰, 20번째 특성 맵에서는 고양이 형태 X.

40번째 계층에서는 전혀 찾아볼 수 없음.

=> 출력층에 가까울수록 원래 형태보다 이미지 특징만 전달됨.

## 5.5 그래프 합성곱 네트워크

그래프 데이터를 위한 신경망.

- 그래프란?: 방향성이 있거나 없는 에지로 연결된 노드의 집합.
- 그래프 신경망
  1. 인접 행렬(adjacency matrix): 노드  $n$ 개  $n \times n$  행렬,  $A_{ij}$ 와 같이 표현.
  2. 특성 행렬(feature matrix): 각 행은 선택된 특성에 대해 각 노드가 갖는 값을 의미
- 그래프 합성곱 네트워크(GCN): 이미지에 대한 합성곱을 그래프 데이터로 확장한 알고리즘, 리드아웃(특성 행렬을 하나의 벡터로 변환하는 함수, 특성 벡터에 대해 평균을 구하고 그래프 전체를 표현하는 하나의 벡터 생성)+ 그래프 합성곱층(그래프 형태의 데이터를 행렬 형태의 데이터로 변환하여 딥러닝 알고리즘 적용할 수 있게 해줌)

### ▼ 그림 5-52 그래프 합성곱 네트워크

